Junior Deogracias MAMBO MOTSOU

Ingénieur logiciel embarqué

motsoujunior17@gmail.com | +33 6 01 16 86 06 | France | Disponible immédiatement
teo1747.github.io/Portfolio | github.com/teo1747 | linkedin.com/in/teo1747

Profil

Ingénieur logiciel embarqué (Polytech Dijon, 2026) spécialisé en firmware C, systèmes d'exploitation temps réel et outillage associé. Chez FORVIA, j'ai développé et débogué du firmware pour microcontrôleurs automobiles ainsi qu'une chaîne de pilotage multithread Python/Kotlin pour un démonstrateur d'éclairage. Dans mes projets personnels, je développe un RTOS, un OS x86-64 et une chaîne de compilation C, avec un soin particulier pour le build, la CI et les tests.

Compétences techniques

Langages	C (C99/C11), Python, C++ (Qt), Kotlin ; assembleur : x86-64 (NASM), Arm Thumb, RISC-V
RTOS & OS	FreeRTOS (tâches, queues, mutex, event groups, stream buffers), Zephyr (devicetree, Kconfig), bare-metal ; noyau : ordonnancement, commutation de contexte, pagination, interruptions, syscalls
Linux	Ubuntu au quotidien ; drivers en espace utilisateur (spidev, i2c-dev, GPIO sysfs) sur Raspberry Pi
Build & toolchains	GCC (arm-none-eabi, cross x86_64-elf), Clang, scripts d'édition de liens, CMake, Ninja, GNU Make, West, Kconfig ; IAR EWARM, MPLAB X (XC16/XC32)
Debug & tests	GDB (distant, QEMU), J-Link, QEMU ; tests unitaires sous ASan/UBSan, tests de conformité et tests différentiels contre des modèles de référence
Automatisation	Git (workflow par pull requests), CI GitHub Actions, outillage Python
Interfaces	UART, SPI, I²C, LIN, MeLiBu, Bluetooth série avec protocole JSON, clients REST/HTTP
Applications	Python multithread (PyQt5, OpenCV), Qt/C++, Android (Kotlin, Jetpack Compose)

Expérience professionnelle

Ingénieur systèmes embarqués — contrat de professionnalisation

sept. 2025 – sept. 2026

FORVIA – Faurecia Interior Systems département Surface Activation, France

Démonstrateur d'éclairage d'ambiance intérieur (responsable logiciel)

- Extension du logiciel de pilotage d'un buck de démonstration intérieur automobile, de quelques afficheurs LED à neuf gateways UART STM32 pilotant des rubans WS2812 répartis en plusieurs zones.
- Application Python/PyQt5 multithread : décodage des animations MP4 avec OpenCV, correspondance entre identifiants LED et pixels de la frame, envoi de chaque frame par un thread par gateway à 921 600 bauds, synchronisés par une barrière ; trames série propriétaires (ASCII avec compression delta).
- Application tablette Android (Kotlin, Jetpack Compose) qui pilote zones, couleurs et presets de luminosité en Bluetooth série, via un protocole de commandes JSON avec codes d'erreur ; mode de secours au clavier.
- Adaptation minimale du firmware des gateways STM32 (filtrage par décalage d'ID si deux gateways partagent une UART).

Environnement : Python 3, PyQt5, OpenCV, Kotlin/Jetpack Compose, STM32 (C), WS2812, UART, Bluetooth SPP, Git

Mise en route firmware : contrôleur d'élément chauffant piloté en LIN (TCU)

- Développement du firmware C sur Infineon TLD4020-3ET (Arm Cortex-M23 avec LIN intégré) : commandes ON/OFF par LIN, PWM de chauffe via le périphérique LDRV, régulation de température par CTN, protections, architecture en machine d'états.
- Diagnostic des blocages de mise en route : HardFault à l'initialisation causé par une révision de silicium différente sur la carte d'évaluation (registre absent) ; perte du debug liée au cavalier de boot et à l'alimentation USB.
- Le pont diviseur de la CTN étant alimenté par la sortie de chauffe commutée, le nœud ADC est un signal carré à 100 Hz pendant la PWM : échantillonnage synchronisé sur le temps haut (ISR à mi-impulsion, moyenne sur 16 mesures), qui a ramené la dispersion dans les limites du besoin de régulation.

Environnement : carte TLD4020-3STD_EVAL, IAR Embedded Workbench, Infineon MCU Configuration Wizard, J-Link embarqué, printf sur LIN (MobaXterm, 19 200 bauds 8N1)

Portage du driver LED MeLiBu : Melexis MLX81116 vers MLX81143

- Portage d'un driver MeLiBu 1.x existant sur RP2040 vers le MLX81143 : encodage de sous-adresse en mode étendu (14 LED par nœud), encodage PWM linéaire 15 bits vérifié sur datasheet et sur matériel ; panneau 22 × 7 (154 LED RVB).
- Correction des défauts de timing bus : génération du break en attente active sous interruptions masquées, section critique réduite au seul break pour ne plus perdre d'octets en réception UART, vidage de la FIFO TX avant chaque trame pour éviter les collisions ; correction de la broche de retour bus.

Environnement : RP2040, Raspberry Pi Pico SDK, machines d'états PIO, C, MeLiBu 1.x

Stagiaire systèmes embarqués

avr. – août 2025

KMUTT — MDT Lab Bangkok, Thaïlande

- Traitement d'image pour un projet d'agriculture de précision : classification de fruits par transfer learning sur VGG16 (Keras/TensorFlow), 98,4 % de bonnes classifications sur un jeu de test de 4 599 images (Fruits-360, 131 classes), et chaîne de détection OpenCV par contours et couleurs.

Environnement : Python, TensorFlow/Keras, OpenCV

Projets personnels et académiques

Personnels

En cours. Je définis l'architecture, les spécifications et la stratégie de vérification ; l'implémentation est assistée par des outils de programmation IA. Validation à ce stade en émulation (QEMU, EmbSIM), pas encore sur cartes physiques.

EmbLinkRTOS — RTOS préemptif pour microcontrôleurs personnel — depuis oct. 2026
github.com/EmbLinkOs/EmbLinkRTOS

- Ordonnanceur préemptif à priorités fixes (bitmap de listes prêtes, FIFO à priorité égale) ; port Cortex-M (commutation PendSV/PSP, sauvegarde FPU paresseuse, sections critiques BASEPRI), ports RISC-V et AVR.
- Sémaphores, mutex avec héritage et plafond de priorité, timeouts ; tests de conformité et différentiels contre un modèle Python de l'ordonnanceur.
- Build CMake/Kconfig avec définitions de cartes (NUCLEO-F446RE, Arduino Uno, ESP32-C3) ; CI GitHub Actions avec ASan/UBSan/TSan, exécutions QEMU et EmbSIM.

EmbLinkOS — système d'exploitation x86-64 écrit de zéro personnel — depuis mai 2026
github.com/EmbLinkOs/EmbLinkOs

- Bootloader BIOS en deux étages en NASM (mode réel, protégé, puis long) et chargeur UEFI PE32+ ; noyau higher-half en C : pagination 4 niveaux, APIC/IO-APIC, SMP, processus ring 3, appels système, chargeur ELF.
- Système de fichiers B-tree copy-on-write (CRC32C, snapshots, AES-XTS) vérifié contre un oracle Python ; FAT32, VFS ; drivers PCI, AHCI, NVMe et USB ; portages de newlib et TCC.
- Autotests noyau, tests unitaires hôte sous ASan/UBSan, tests d'intégration pilotés sous QEMU ; debug GDB distant.

EmbCC — chaîne de compilation croisée C personnel — depuis juil. 2026
github.com/EmbLinkOs/EmbCC

- Compilateur C99, assembleur et éditeur de liens avec back-ends Cortex-M, RISC-V, AVR et x86-64 ; utilitaires firmware (génération d'en-têtes/startup depuis SVD, empaquetage HEX/UF2).
- Vérification : encodages comparés à llvm-mc/GNU as, code généré exécuté sous QEMU et comparé à GCC/Clang ; CI GitHub Actions sous Linux et macOS avec GCC et Clang.

Terminal de contrôle d'accès RFID/PIN sous FreeRTOS personnel — oct. 2026
github.com/teo1747/stm32-freertos-access-control

- Drivers au niveau registre (sans HAL constructeur) sur STM32F446RE : RFID MFRC522 en SPI, SSD1306, RTC DS3231 et EEPROM 24LC256 en I²C, clavier, servo en PWM, watchdog ; six tâches FreeRTOS avec queues, mutex, event groups et stream buffers ; PIN hachés SHA-256 salés et étirés ; journal protégé par CRC.
- Tests unitaires hôte avec sanitizers et scénarios système sous EmbSIM ; pas encore testé sur carte physique.

Environnement : NUCLEO-F446RE, FreeRTOS 11, CMSIS, arm-none-eabi-gcc, CMake, Ninja

Polytech Dijon

Thermomètre temps réel : PIC24 bare-metal, puis FreeRTOS sur PIC32MZ Polytech, seul puis en binôme — 2025 – 2026
github.com/teo1747/pic24-32-rt-thermometer

- V1 (seul) : PIC24FJ1024GB610 en bare-metal, horodatage RTCC et journalisation en EEPROM SPI. V2 (binôme) : chaîne FreeRTOS de tâches acquisition, analyse et affichage sur PIC32MZ2048EFH100, reliée par des queues et des sémaphores binaires ; messages horodatés ; bascule °C/°F par bouton.

Environnement : Explorer 16/32, MPLAB X, XC16/XC32, Harmony v3/MCC, FreeRTOS 10.5, capteur TC1047A, débogueur embarqué

Étude de faisabilité Zephyr RTOS sur STM32 TP Polytech — 2025 – 2026
github.com/teo1747/zephyr-stm32-sensor-hub

- Trois threads se partagent un bus I²C : BMP280 (température/pression), MPU6050 (centrale inertielle 6 axes) et OLED SSD1306 via l'API display et CFB ; données partagées protégées par mutex ; carte décrite en overlay devicetree (dépôt reconstruit d'après le rapport).

Environnement : NUCLEO-G474RE, Zephyr, West, CMake, Ninja, STM32CubeProgrammer, console série

Périphériques sous Linux embarqué sur Raspberry Pi Polytech, équipe — mars 2025

- Drivers C en espace utilisateur pour modules Pmod : capteur de lumière SPI via spidev (mode 3, 2 MHz), capteur d'humidité I²C via i2c-dev, GPIO via sysfs ; compilation par Make.

Formation & langues

Diplôme d'ingénieur (Bac+5) — Systèmes embarqués — 2023 – sept. 2026
Polytech Dijon Dijon

Diplômé. Dernière année en alternance chez FORVIA. Enseignements : électronique, PCB, temps réel, SoC/FPGA.

Classes préparatoires (2 ans) — mathématiques, physique, électronique — 2021 – 2023
IUC Cameroun

Langues Anglais : langue maternelle (anglophone), TOEIC C1 (2024) | Français : courant (B2)
Certification Cisco CCNA 1 (juillet 2024)